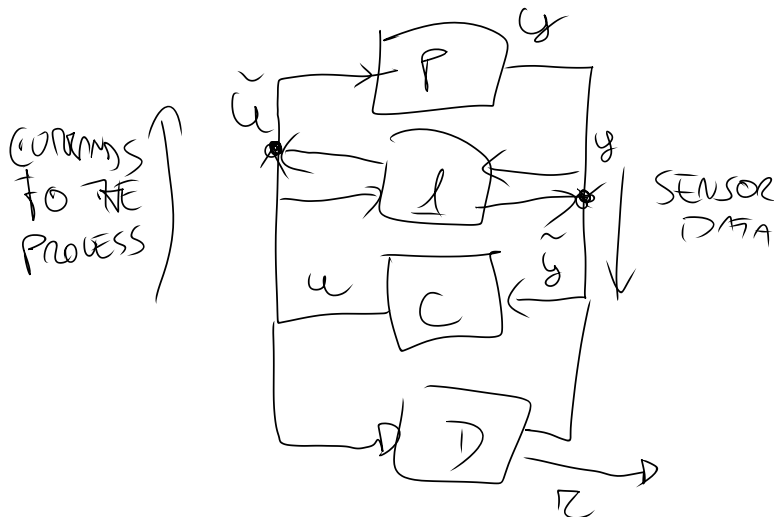


Today we take a look at basic modelling of basic attacks, then we proceed with the solution of state space equations.



Now, let's take a look at denial of service attacks (DoS).

They're attacks in which the attacker prevents sensor data or commands from reaching the respective destination. Thus, sensor data will not reach the controller, and commands will not reach the process.

Basically, it's an attack on Availability.

Let's focus on a DoS attack on the commands.

To do this, let's take a look at the data deception equation.

We're going to model the relation between u and $\tilde{u}$.

We've already seen that:

$$\tilde{u} = u + \Gamma_I^u \cdot b^u$$

Remember that Γ_I^u is a selection matrix, as the attack targets just a subset of possible commands (like attacking 3 out of possible ten commands).

We have something like this:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} \cdot \\ \cdot \\ \cdot \\ \cdot \\ \cdot \end{pmatrix}$$

where the rows are given by total inputs, and columns by attack resources.

Let's see now how to model DoS attack on u using data deception equation.

In practical terms, a DoS is not a bias injection attacks. It could be performed in

several ways, like the attacker cutting a cable, etc.

To model it mathematically, however, we model it as a bias injection attack.

In practice, what happens? When a process does not receive commands, he might apply the last received commands. There are multiple other strategies, however. Call $u(\tau)$ the last command received by the plant before the attack being performed.

$$\tilde{u}(t) = u(t) + \Gamma_I^u \cdot b^u(t)$$

Let's define how the bias $b^u(t)$ is structured in a DoS attack.

$$\begin{cases} b^u(t) = S \cdot (\Gamma_I^u)^T \cdot (u(\tau) - u(t)) \\ \tilde{u}(t) = u(t) + \Gamma_I^u \cdot b^u(t) \end{cases}$$

Let's start by taking a look at the term $u(\tau) - u(t)$.

We have:

$$\begin{pmatrix} u_1(\tau) \\ u_2(\tau) \\ \vdots \\ u_{n_u}(\tau) \end{pmatrix} - \begin{pmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_{n_u}(t) \end{pmatrix}$$

So, τ is the start of the attack, and t is a time s.t. $t > \tau$.

Then, we also have the transpose of the selection matrix $(\Gamma_I^u)^T$ as our attack is not targeting all the control.

Then, let's happen the transpose to the subtraction:

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \cdot \left[\begin{pmatrix} u_1(\tau) \\ u_2(\tau) \\ \vdots \\ u_{n_u}(\tau) \end{pmatrix} - \begin{pmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_{n_u}(t) \end{pmatrix} \right] = \begin{pmatrix} u_1(\tau) - u_1(t) \\ u_3(\tau) - u_3(t) \\ u_5(\tau) - u_5(t) \end{pmatrix}$$

We focus our attack only on variables which are actually under attack.

S matrix is a matrix in function of time. It's a start/stop matrix, used to stop the attack. It's a matrix of all zeros except the diagonal, which could contain either 0 or 1, depending on the fact he wants to start the attack or not.

$$b^u(t) = \underbrace{\begin{pmatrix} . & 0 & 0 \\ 0 & . & 0 \\ 0 & 0 & . \end{pmatrix}}_S \cdot \begin{pmatrix} u_1(\tau) - u_1(t) \\ u_3(\tau) - u_3(t) \\ u_5(\tau) - u_5(t) \end{pmatrix}$$

With this matrix we can model intermittent denial of service attacks, allowing to attack variables independently.

So, despite the DoS not being a bias injection, we can model it mathematically in this way.

What about the attacker resources?

- No model knowledge;
- No disclosure resources;
- Disruption resources.

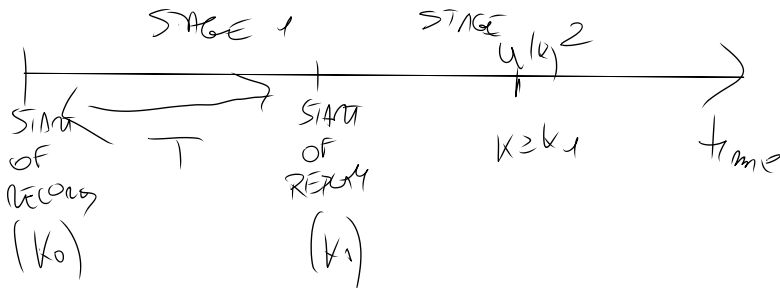
It can be quite difficult to detect as an attack, as it could be understood as a communication problem or a fault.

Now, let's take a look at **Replay attack**. It's a two-stages attack.

For stage 1, we have the record stage. The attacker records some sensor data and/or command data. So, the attacker is recording some measurements $y(t)$ and/or $u(t)$.

For stage 2, we have the replay stage plus attack. In this phase, the attacker replaces the real time data with the recorded one

Typically, in parallel, another attack is performed, like a physical attack.



In stage 1 we just accumulate data.

$$l_k = l_{k-1} \cup \begin{pmatrix} Y^y & 0 \\ 0 & Y^u \end{pmatrix} \cdot \begin{pmatrix} y \\ u \end{pmatrix}, l_{k_0} = \{\}$$

Remember that Y^y and Y^u are selection matrices of disclosure resources.

Then, for stage 2.

$$b_k^u = (\Gamma_I^u)^T \cdot (u(k-T) - u(k))$$

What is T ? It's the duration of the record stage. We record T seconds of data. Fresh controls are replaced with controls taken at $k - T$.

It's a difficult attack to detect because fresh data are replaced not with strange/random data, but with data coming from the same control system working in normal operations.

Also, mathematically it's similar to a delay impacting on communications.

The attack resources? (basic version)

- No model knowledge
- Disclosure resources (stage 1)
- Disruption resources (stage 2)

We're also going to study zero dynamics attacks. Let's recap the definitions:

For these kinds of attacks, the idea is to inject false data in u , in order to have no impact on the measurements $y(t)$, but a large impact on the state $x(t)$.

Also, for $y(t)$, we have that it's typically a subset of x , and x is typically a large vector of variables.

They require model knowledge, disruption resources.

The last category of attacks is given by Covert attack. Attackers manipulate $u(t)$ to push the plant outputs to a different reference value, decided by the attacker.

At the same time, some measurements are altered, to covert the attack.

Note: to get data, attackers could do a cyber attack on the sensors, but also could install sensors himself. In the same way, to modify the controls the attacker could install actuators.

Let's see now notation on different types of attacks.

First, we have open-loop attacks: they are performed without using any real time information from the system (no disclosure resources).

Then, we have closed-loop attacks, through which attacks are decided based on the processing (attack law) of real time data coming from the system.

They use disclosure resources.

The second difference is between static and dynamic attacks.

Let's see a quick anticipation of the **solution of state space equation in**

continuous time. A physical process can be modelled with state space model.

We've see how for a process the first equation is the state space equation, and the other is the output equation.

$$\begin{cases} \dot{x} = f(x, u) \\ y = g(x, u) \end{cases} \quad \text{general non linear system}$$

If we have a linear system, equations are simpler and are given by the standard equations:

$$\begin{cases} \dot{x} = \underbrace{Ax + Bu}_{f(\cdot)} \\ y = \underbrace{Cx + Du}_{g(\cdot)} \end{cases}$$

Imagine a system in which we apply an input u , and have an output y . We start our experiment from time t_0 , and from then we start applying a given input and seeing the answer. The state will also change over time, and what will it be its evolution?

Remember that $\dot{x}$ denotes an implicit model, telling how the variables are increasing and decreasing over time. We don't want to know just about the derivative $\dot{x}$, and we want the whole solution x .

Measurements are related to detection, and state is related to the impact of the attack.

So, we want to measure the effect on the state and measurements when giving a specific input.

In general, the solution can be written in an abstract way like this:

We have an initial time t_0 , and a time $t > t_0$, and we want to measure the state and measurements at time t .

$$x(t) = \Phi(t, t_0, x(t_0), u_{[t_0, t]})$$

So, $x(t)$ is a function of the time we want to compute the state t , the initial time t_0 , the initial state $x(t_0)$, and the entire input sequence between t_0 and t .

Similarly for the output:

$$y(t) = \gamma(t, t_0, x(t_0), u_{[t_0, t]})$$

Φ is called the state transition function, telling how the state transits from the value at t_0 to t , and γ is the output transition function.

Let's finish with one example.

Our model is a tank. We want to control the water level $h(t)$ of a tank by acting of the inflow of water.



$u(t)$ is the flow of water. The variable we want to control $y(t) \equiv h(t)$, so the level of water.

We have $V(t) = S \cdot h(t)$. The relation with h is in the derivative, as

$V(t) = S \cdot \dot{h}(t) = u(t)$. This is the conservation of mass.

The state variable is h , as it's the only variable we differentiate.

Thus, $x_1(t) = h(t)$.

This leads us to:

$$\begin{cases} \dot{x} = 0x + \frac{1}{S}u \\ y = x + 0u \end{cases}$$

The solution is this:

$$x(t) = x(t_0) + \frac{1}{S} \int_{t_0}^t u(z) dz$$

By starting our experiment at t_0 , imagine we have a flow of water of $u(t)$. If we want to compute the level at any time t , what is the key information we must have to compute the level at time t ? The initial value $x(t_0)$. This tells us that in order to compute any future state or output, we need to know the initial state.

Thus, the state of a dynamical system is the set of all information to be able to compute the future evolution of the system, given an input.

What is then a dynamical system? It's a system that has a state, quite simply. In practical terms though, if we have a system and we specify an input, all the information of the input is not enough to tell us what is the future evolution of the system. Given one input, we can have infinite many possible outputs.

Once we specify the initial state, however, for a given input we can have only one output.

Dynamical systems are also defined as systems with memory.

The opposite are given by static systems, systems with none state. For one input, there is only one output. The output at any time is a function of the input, $y(t) = g(u(t))$.

One example is given by a spring, fixed on a wall, and on this we apply a force F to control the extension y . We have $y = -\frac{1}{K} \cdot u$.

These are memory-less systems.

Finally, let's take a look at the distinction of static and dynamic attacks:

- In static attacks, the attack law is a static system. Current attacks will depend only on current information.
- Dynamic attacks, instead, sees the attack law as a dynamical system.

In static systems, the equation says that the current output is a function only of the current control. In dynamical systems, the current output or state is a function of the initial state and the entire input from t_0 to t .